

Visitor Management - Backend (guide simple)

Ce document explique ce que fait le backend de la gestion des visiteurs, simplement, sans jargon. Si tu veux le détail de l'API, ouvre Swagger (voir § 5).
Simple guide - what the Visitor Management backend does, how its data is organized, and how to run it.

Projet : Sustainable Farm - ferme de mangues séchées, Burkina Faso -> Allemagne (Infineon × BIT Excellence Program) Module : Visitor Management (développé par Alix Carine VEBAMBA) Assistant technique : opencode

1. En bref

Le backend couvre tout le parcours d'un visiteur : choix d'un créneau de visite, inscription, confirmation avec email, briefing sécurité, visite guidée, feedback, et participation à des événements. C'est un outil interne pour l'équipe de la ferme (guides, accueil), pas une application grand public.

- Technologie : Spring Boot (Java 21), base PostgreSQL, API REST.
- Fiabilité : 222 tests automatisés, 0 échec.
- Emails : la confirmation de réservation et le rappel 24 h avant sont réellement envoyés (SMTP), ou simplement affichés en console en phase de développement.
- À jour pour le frontend : l'API accepte les appels du site web (CORS) et dispose
- de données d'exemple pour développer immédiatement (profil dev).

2. Ce que fait le module (les écrans fonctionnels)

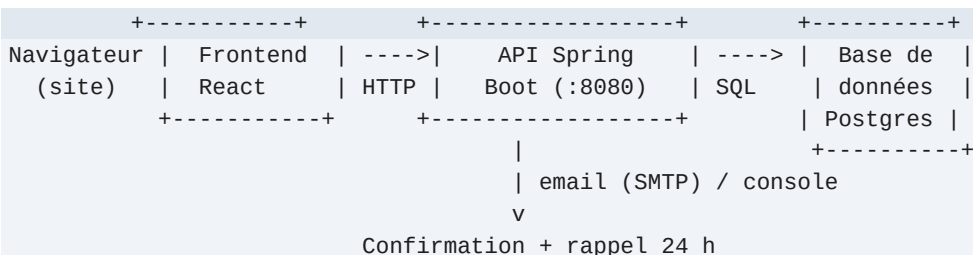
N°	Fonction	En une phrase
1	Dashboard	Vue d'ensemble : visiteurs de la semaine, créneaux occupés, briefings en attente, satisfaction, événements à venir.
2	Planification des visites	L'équipe crée les créneaux (ex. 09 h - 11 h, 14 h - 16 h), avec capacité max 10 personnes, fermé le dimanche.
3	Inscription visiteurs	Formulaire (nom, contact, langue, besoins spéciaux?) + suivi : en attente -> confirmé -> check-in. Détecte les prospects (visiteurs avec intention d'achat).
4	Programme éducatif	Le parcours type de la visite : 6 étapes (accueil -> verger -> irrigation -> solaire -> transformation -> questions). Gestion d' ateliers (solaire, dégustation de mangues, journée écoles).
5	Briefing sécurité	Chaque visite confirmée a un briefing sécurité obligatoire (qui, quand, signature). Obligatoire avant l'accès au site.
6	Réservations agritouristiques	Les activités payantes (visite 5 000 F, dégustation 3 000 F) avec réservation, paiement, confirmation par email et rappel 24 h avant (automatique).
7	Feedback	Après la visite : note de satisfaction, avis sur le briefing, la valeur éducative, la recommandation + envoi d'enquêtes.
8	Événements	Regrouper des visiteurs : porte-ouvertes, visite d'acheteurs, écoles?

En résumé, l'ordre de vie d'un visiteur :

Choix du créneau -> Inscription -> Confirmation email -> Briefing sécurité
-> Visite (programme éducatif) -> Feedback -> (éventuel événement)

3. Comment c'est organisé

Un seul backend, une seule base de données. Le site web (frontend React) viendra ensuite s'y brancher.



- Toutes les fonctions sont regroupées dans le module Visitor Management
- (modules/visitormanagement/), avec le partage technique dans core/.
- L'interface Web de l'API (Swagger) est automatiquement générée : c'est la meilleure
- façon de voir chaque écran -> chaque URL -> chaque donnée.

4. Les données (schéma et diagrammes de classes)

Clés de lecture des diagrammes :

Un A concerne plusieurs B :			Un A correspond à un seul B :		
+-----+ 1	*	+-----+	+-----+ 1	1	+-----+
A	----->	B	A	---->	B
+-----+		+-----+	+-----+		+-----+

4.1 Planification, inscription, briefing

C'est le cœur : qui vient, quand, et qui a reçu le briefing sécurité.

+-----+	+-----+	+-----+
TimeSlot 1	Registration 1	Briefing
(créneau de visite)	(inscription)	(briefing sécurité)
+-----+	+-----+	+-----+
id	id	id
date	visitor (FK)	registration (FK)
startTime	timeSlot (FK)	deliveredAt
endTime	eventId (FK)	staffMember
maxCapacity (10)	visitPurpose	signature
status	isProspect (prospect)	status
guideId	status	+-----+
+-----+	+-----+	
1		
*	1..*	
+-----+		
Visitor	<-----+	
(visiteur / groupe)		
+-----+		
id		
fullName	Relations :	
groupSize	? 1 créneau -> plusieurs inscriptions	
email / phone	? 1 visiteur -> plusieurs inscriptions	
language	? 1 inscription -> 1 briefing	
type (individu/		
groupe / école /		
partenaire)		
specialNeeds		
+-----+		

4.2 Réservations agritouristiques

Le catalogue d'activités payantes et les réservations, avec leurs emails.

+-----+	+-----+
AgriActivity 1	Booking
(offre / activité)	(réservation)
+-----+	+-----+
id	id
name	activity (FK)
price (FCFA)	timeSlot (FK)
capacity	visitorFullName
durationMinutes	visitorEmail
description	visitorPhone
active	peopleCount
+-----+	totalAmount
	status / paymentStatus
	paymentMethod

```

| reminder (rappel 24h) |
| confirmation/reminder |
| emails envoyés |
+-----+

```

Une réservation réserve aussi un créneau de la journée (le TimeSlot de la partie 4.1).

4.3 Programme éducatif

Le parcours de visite et les ateliers. Deux listes indépendantes (pas de lien entre elles).

TourStop	Workshop
(étape de la visite)	(atelier programmé)
id	id
name	name
position (1 à 6)	durationMinutes
description	targetGroup (public)
durationMinutes	facilitator (animateur)
maxCapacity	description
location / demo	status (brouillon / actif)
safetyNotes	
active	

Les 6 étapes actuelles : 1) Accueil & briefing 2) Verger de manguiers 3) Irrigation goutte-à-goutte 4) Centrale solaire & tracking 5) Unité de transformation 6) Questions finales.

4.4 Feedback et enquêtes

Le visiteur répond à une enquête (SurveySend), qui produit un feedback.

Visitor	1	SurveySend	1 1	Feedback
(visiteur / groupe)		(enquête envoyée)		(réponse du v.)
id		id		id
visitor (FK)		visitor (FK)		visitor (FK)
channel (email / ...)		surveySend (FK)		surveySend (FK)
messageTemplate		origin / channel		origin / channel
sentAt		rating (note 1-5)		rating (note 1-5)
status		briefingClear		briefingClear
		educationalValue		educationalValue
		recommend		recommend
		comment		comment
		routedTo / submittedAt		routedTo / submittedAt

4.5 Événements

Un événement regroupe plusieurs inscriptions (d'où le champ eventId dans Registration).

Event	1	*	Registration
(événement programmé)			(voir 4.1 - le champ eventId
id			relie une inscription à un
title			événement)
type (porte-ouvertes /			
acheteur / école /			
communauté)			
startDateTime / end			
maxCapacity			
location / description			
status (brouillon /			
publié / annulé / ...)			

+-----+

4.6 Récapitulatif des relations

Entité	Rôle	Liens
TimeSlot	Créneau de visite (date/heure/capacité)	-> plusieurs `Registration` et `Booking`
Visitor	Visiteur ou groupe	-> plusieurs `Registration`, `Feedback`, `SurveySend`
Registration	Inscription à un créneau (ou événement)	-> 1 `Visitor`, 1 `TimeSlot`, 1 `Briefing`, option `Event`
Briefing	Preuve du briefing sécurité	-> 1 `Registration`
Event	Événement regroupant des visiteurs	-> plusieurs `Registration`
AgriActivity	Activité payante du catalogue	-> plusieurs `Booking`
Booking	Réservation d'activité + emails	-> 1 `AgriActivity`, 1 `TimeSlot`
TourStop	Étape du parcours de visite	indépendant
Workshop	Atelier (éducatif)	indépendant
SurveySend	Enquête envoyée au visiteur	-> 1 `Visitor`, -> 1 `Feedback`
Feedback	Réponse du visiteur	-> 1 `Visitor`, 1 `SurveySend`

Toutes les entités partagent un identifiant id et les dates createdAt / updatedAt.

5. Accéder à l'API (Swagger)

La meilleure porte d'entrée : l'interface Swagger, générée automatiquement.

- Swagger UI : <http://localhost:8080/swagger-ui/index.html>
- Vérifier que le backend tourne : <http://localhost:8080/actuator/health>

Aperçu des groupes d'URLs (tout est sous /api/v1) :

Ressource	URLs principales
Créneaux	`/time-slots` (+ `/availability`)
Visiteurs	`/visitors`
Inscriptions	`/registrations` (+ `/approve`, `/reject`, `/check-in`, `/cancel`)
Briefings	`/registrations/{id}/briefing` (+ `/deliver`)
Programme éducatif	`/tour-stops`, `/workshops` (+ `/publish`, `/deactivate`)
Réservations	`/activities`, `/bookings` (+ `/pay`, `/confirm`, `/complete`, `/cancel`)
Feedback	`/feedback`, `/feedback/summary`, `/surveys`
Événements	`/events` (+ `/register`, `/publish`, `/registrations`)

6. Démarrer

Prérequis : Docker. Copier .env.example vers .env, puis :

```
docker compose up -d --build
# Backend : http://localhost:8080
# Swagger : http://localhost:8080/swagger-ui/index.html
```

Options utiles (dans .env)

Variable	À quoi ça sert	Exemple
SPRING_PROFILES_ACTIVE	Charge des **données d'exemple** (stops, ateliers, activités, créneaux, un événement et un visiteur) pour développer. **Jamais en production.**	`dev`
APP_CORS_ORIGINS	Autorise le site web (frontend) à appeler l'API.	`http://localhost:3000,http://localhost:5173`
MAIL_ENABLED	`true` -> envoi réel des emails (SMTP) ; `false` -> affichage en console.	`false`
MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM	Paramètres du serveur SMTP.	`smtp.orange.bf` / `587`

7. Qualité

- 222 tests automatisés, 0 échec : règles de gestion (capacités, statuts, paiements),
- emails (confirmation, rappel 24 h, échec d'envoi), toutes les URLs et le schéma de données.
- Relancer les tests (via Docker, aucun JDK local nécessaire) :

```
docker run --rm -v "$PWD/backend":/src -w /src -v "$HOME/.m2":/root/.m2 \
maven:3.9-eclipse-temurin-21 ./mvnw test
```

8. Et maintenant

Le backend est terminé et prêt. Reste à construire le frontend (les écrans portent sur les 8 fonctions du § 2). La base de données contiendra les données d'exemple dès qu'on lance avec `SPRING_PROFILES_ACTIVE=dev`, donc le site pourra être développé et testé immédiatement.

Dernière mise à jour : 2026-09-10